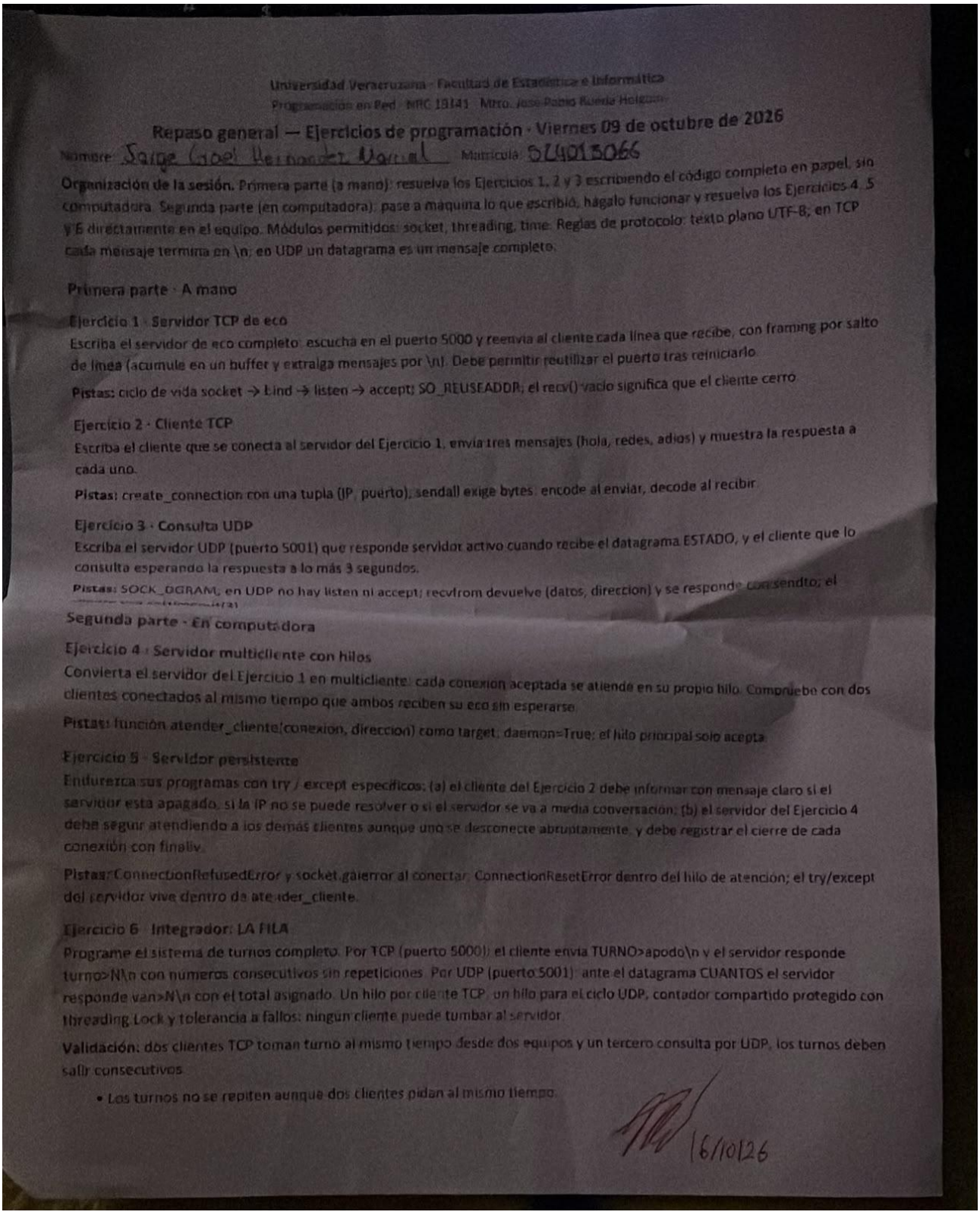
Jorge Gael Hernández Marcial



Universidad Veracruzana · Facultad de Estadística e Informática
Programación en Red · NRC 19141 · Mtro. José Pablo Rueda Holguín

Repaso general — Ejercicios de programación · Viernes 09 de octubre de 2026

Nombre: Jorge Gael Hernandez Marcial Matrícula: S24013066

Organización de la sesión. Primera parte (a mano): resuelva los Ejercicios 1, 2 y 3 escribiendo el código completo en papel, sin computadora. Segunda parte (en computadora): pase a máquina lo que escribió, hágalo funcionar y resuelva los Ejercicios 4, 5 y 6 directamente en el equipo. Módulos permitidos: socket, threading, time. Reglas de protocolo: texto plano UTF-8; en TCP cada mensaje termina en \n; en UDP un datagrama es un mensaje completo.

Primera parte · A mano

Ejercicio 1 · Servidor TCP de eco

Escriba el servidor de eco completo: escucha en el puerto 5000 y reenvía al cliente cada línea que recibe, con framing por salto de línea (acumule en un buffer y extraiga mensajes por \n). Debe permitir reutilizar el puerto tras reiniciarlo.

Pistas: ciclo de vida socket → bind → listen → accept; SO_REUSEADDR; el recv() vacío significa que el cliente cerró.

Ejercicio 2 · Cliente TCP

Escriba el cliente que se conecta al servidor del Ejercicio 1, envía tres mensajes (hola, redes, adios) y muestra la respuesta a cada uno.

Pistas: create_connection con una tupla (IP, puerto); sendall exige bytes: encode al enviar, decode al recibir.

Ejercicio 3 · Consulta UDP

Escriba el servidor UDP (puerto 5001) que responde servidor activo cuando recibe el datagrama ESTADO, y el cliente que lo consulta esperando la respuesta a lo más 3 segundos.

Pistas: SOCK_DGRAM; en UDP no hay listen ni accept; recvfrom devuelve (datos, dirección) y se responde con sendto; el

Segunda parte · En computadora

Ejercicio 4 · Servidor multicliente con hilos

Convierta el servidor del Ejercicio 1 en multicliente: cada conexión aceptada se atiende en su propio hilo. Compruebe con dos clientes conectados al mismo tiempo que ambos reciben su eco sin esperarse.

Pistas: función atender_cliente(conexion, direccion) como target; daemon=True; el hilo principal solo acepta.

Ejercicio 5 · Servidor persistente

Endurezca sus programas con try / except específicos: (a) el cliente del Ejercicio 2 debe informar con mensaje claro si el servidor está apagado, si la IP no se puede resolver o si el servidor se va a media conversación; (b) el servidor del Ejercicio 4 debe seguir atendiendo a los demás clientes aunque uno se desconecte abruptamente, y debe registrar el cierre de cada conexión con finally.

Pistas: ConnectionRefusedError y socket.gaierror al conectar; ConnectionResetError dentro del hilo de atención; el try/except del servidor vive dentro de atender_cliente.

Ejercicio 6 · Integrador: LA FILA

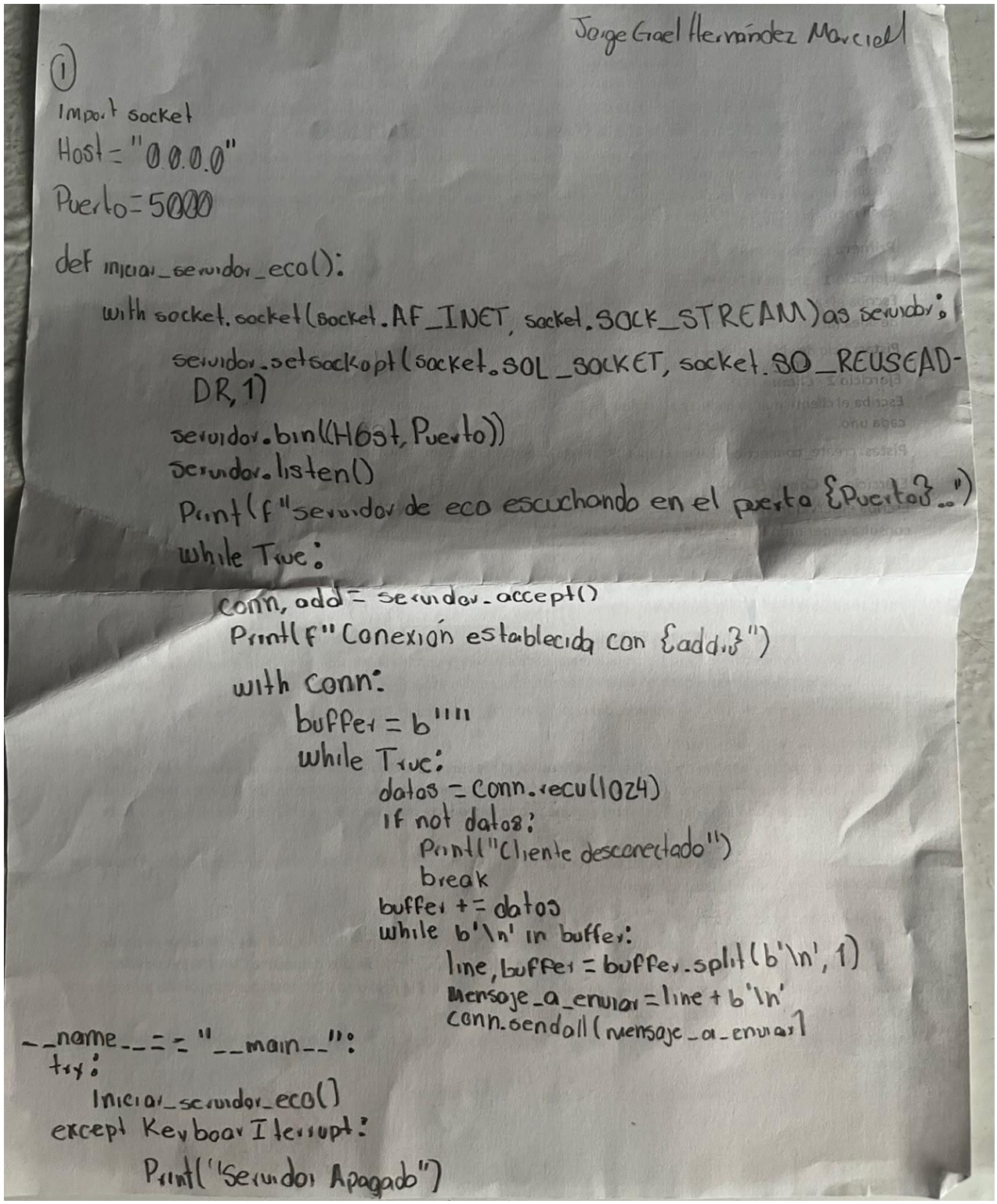
Programe el sistema de turnos completo. Por TCP (puerto 5000): el cliente envía TURNO>apodo\n y el servidor responde turno>N\n con números consecutivos sin repeticiones. Por UDP (puerto 5001): ante el datagrama CUANTOS el servidor responde van>N\n con el total asignado. Un hilo por cliente TCP, un hilo para el ciclo UDP, contador compartido protegido con threading.Lock y tolerancia a fallos: ningún cliente puede tumbar al servidor.

Validación: dos clientes TCP toman turno al mismo tiempo desde dos equipos y un tercero consulta por UDP, los turnos deben salir consecutivos.

- Los turnos no se repiten aunque dos clientes pidan al mismo tiempo.

16/10/26

Jorge Gael Hernández Marcial



Jorge Gael Hernández Marcial

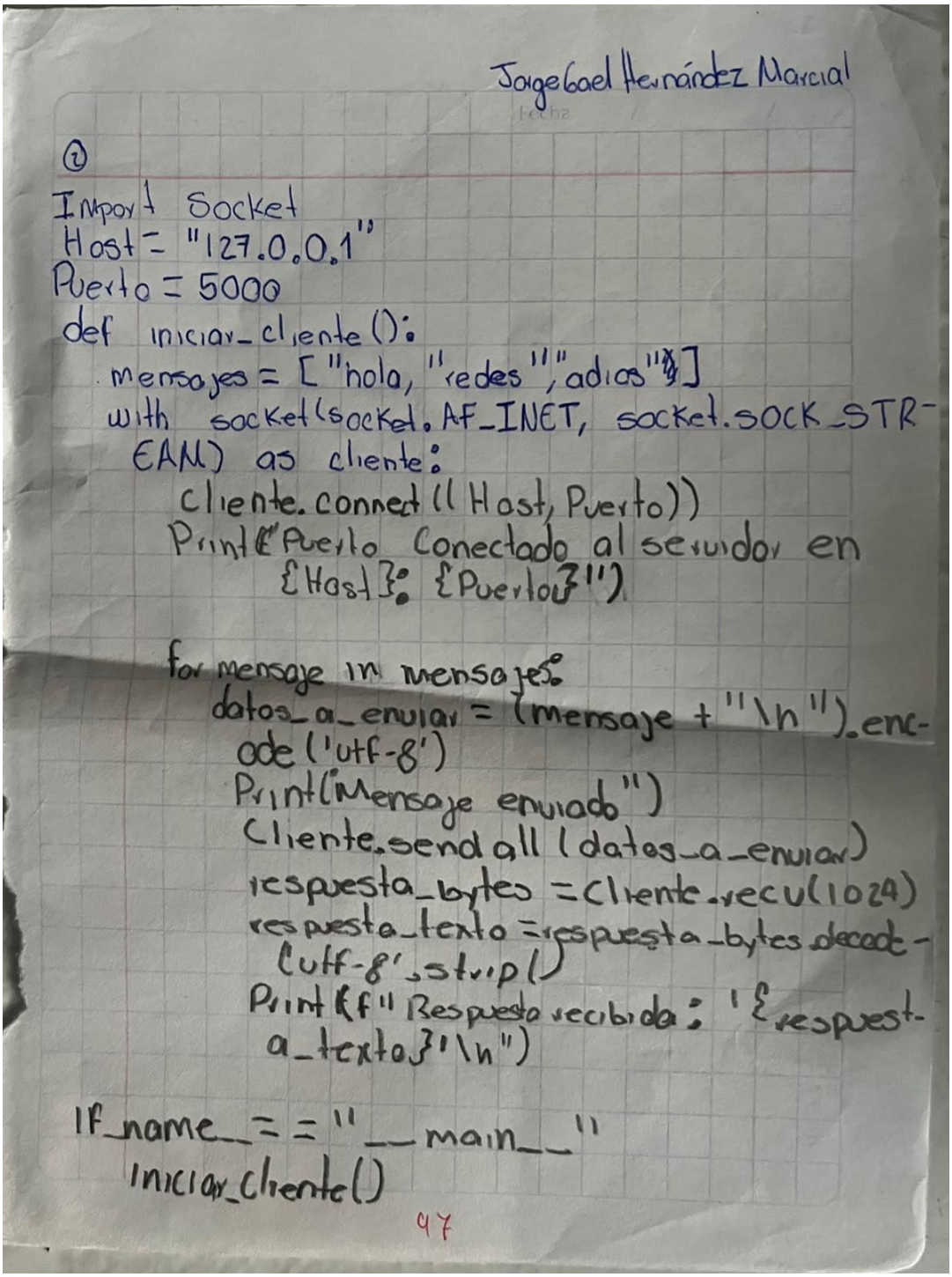
①

```
Import socket
Host = "0.0.0.0"
Puerto = 5000

def iniciar_servidor_eco():
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as servidor:
        servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        servidor.bind((Host, Puerto))
        servidor.listen()
        Print(f"servidor de eco escuchando en el puerto {Puerto}...")
        while True:
            conn, addr = servidor.accept()
            Print(f"Conexión establecida con {addr}")
            with conn:
                buffer = b""
                while True:
                    datos = conn.recv(1024)
                    if not datos:
                        Print("Cliente desconectado")
                        break
                    buffer += datos
                    while b'\n' in buffer:
                        line, buffer = buffer.split(b'\n', 1)
                        mensaje_a_enviar = line + b'\n'
                        conn.sendall(mensaje_a_enviar)

__name__ == "__main__":
    try:
        Iniciar_servidor_eco()
    except KeyboardInterrupt:
        Print("Servidor Apagado")
```

Jorge Gael Hernández Marcial



Jorge Gael Hernández Marcial

Fecha

②

```
Import Socket
Host = "127.0.0.1"
Puerto = 5000
def iniciar_cliente():
    mensajes = ["hola, "redes", "adios"]
    with socket(socket.AF_INET, socket.SOCK_STR-
    EAM) as cliente:
        cliente.connect((Host, Puerto))
        Print(f"Puerto Conectado al servidor en
            {Host}: {Puerto}")

        for mensaje in mensajes:
            datos_a_enviar = (mensaje + "\n").enc-
            ode('utf-8')
            Print("Mensaje enviado")
            Cliente.sendall(datos_a_enviar)
            respuesta_bytes = Cliente.recv(1024)
            respuesta_texto = respuesta_bytes.decode-
            (utf-8', strip()
            Print(f"Respuesta recibida: '{respuest-
            a_texto}'\n")

If __name__ == "__main__"
    iniciar_cliente()
```

97